grblHAL auf FYSETC R4: Flashen & Konfiguration

Oct 19, 2026 @Philipp | Status: Aufgebaut & getestet (Endkonfiguration vom 19.10.2026)

1. Überblick

Dein Board ist ein **FYSETC R4 V1.0** (Aufdruck „R4_V1 2022/04/11“). Es hat einen Raspberry Pi RP2040 als Prozessor und 4 fest verbaute TMC2209-Treiber im UART-Modus. FYSETC baut es eigentlich für den Voron V0 mit Klipper. Darauf läuft **grblHAL**, das den RP2040 ausgereift unterstützt. Eine fertige Board-Datei für das R4 gab es nicht – die habe ich aus dem Schaltplan von FYSETC geschrieben und die Firmware gebaut (Datei `grblHAL-FYSETC-R4.uf2`).

Das Ziel-Setup: X mit einem Motor und Endstop, Z mit zwei Motoren und je einem Endstop (Squaring: jeder Motor stoppt beim Homing an seinem Schalter), dazu ein RC-Servo, angesteuert per `M280`. Der zweite Z-Motor läuft über den E-Treiber, sein Endstop über den E-STOP-Eingang. Der Y-Treiber bleibt frei.

Funktion	Anschluss am R4	GPIO	TMC-Adresse
X-Motor	MOTOR-X	Step 3, Dir 2, Enable 4	1
Z-Motor 1	MOTOR-Z	Step 19, Dir 18, Enable 17	3
Z-Motor 2	MOTOR-E	Step 6, Dir 5, Enable 7	0
(Y, frei)	MOTOR-Y	Step 16, Dir 15, Enable 14	2
X-Endstop	X-STOP	11	–
Z-Endstop 1	Z-STOP	13	–
Z-Endstop 2	E-STOP	10	–
Servo-Signal	Laser-Stecker	22	–
Probe	PROBE	25	–
Spindel-PWM / -Enable	FAN0 / HEAD	20 / 24	–
Kühlmittel (M8)	BED	23	–
TMC-UART	intern	TX 8, RX 9	–

Die Firmware berücksichtigt die 0,22-Ω-Messwiderstände des R4 (grblHAL rechnet sonst mit 0,11 Ω – der Motorstrom wäre doppelt so hoch wie eingestellt) und die abweichende Reihenfolge der Treiber-Adressen.

Du brauchst:

- FYSETC R4 V1.0, 24-V-Netzteil (12 V geht auch) und ein USB-C-Datenkabel
- 3 mechanische Endstops, 2–3 Schrittmotoren, einen RC-Servo

- Nur bei großen Servos (Blockierstrom über ca. 3 A) oder wenn ein Raspberry Pi am Board hängt: eine separate 5-V-Versorgung (BEC)
- Einen G-Code-Sender am PC, z. B. ioSender (Windows, für grblHAL gemacht) oder gSender / CNCjs / UGS

2. Nachbau: Elektronik & Firmware

Dieses Kapitel führt dich einmal komplett vom Auspacken bis zur Maschine, die referenziert, fährt und den Servo bewegt. Du brauchst dafür kein Vorwissen über grblHAL. Die Mechanik (Rahmen, Riemen, Spindeln) gehört nicht dazu, die Aufbauanleitung kommt später dazu. Alle Werte stammen von der getesteten Maschine. Die Kapitel 3 bis 6 weiter unten erklären dasselbe ausführlicher.

Die Dateien

Datei	Wofür
`grblHAL-FYSETC-R4.uf2`	Fertige Firmware zum Draufkopieren auf das Board. Enthält die Endkonfiguration als Standardwerte und die Servo-Korrektur.
`grblHAL-RP2040-fysetc-r4.patch`	Der Unterschied zum offiziellen grblHAL-Quellcode: Board-Definition, Servo-Korrektur, Standardwerte. Nur nötig, wenn du die Firmware selbst bauen oder ändern willst.
`machine_defaults.h`	Alle Maschinenwerte (Schritte pro mm, Verfahrwege, Geschwindigkeiten, Homing) als lesbare Liste. Steckt im Patch, liegt zum Nachlesen und Bearbeiten auch einzeln bei.
Dieses Dokument	Anleitung, Befehle, Pin-Belegung, Fehlersuche.

Das brauchst du

Teil	Menge	Hinweis
FYSETC R4 V1.0	1	Aufdruck „R4_V1 2022/04/11“. Vier TMC2209-Treiber sind fest verlötet. Andere Platinenversionen habe ich nicht getestet.
Netzteil 24 V Gleichspannung	1	Nicht über 28 V (Schutzdiode). 12 V geht auch. Als Faustwert reichen 3 A, nicht gemessen.
USB-C-Datenkabel	1	Ein reines Ladekabel funktioniert nicht.
Schrittmotoren, 2 Phasen, 4 Adern, 200 Schritte pro Umdrehung	3	Einer für X, zwei für Z. Das Board liefert höchstens 958 mA pro Motor, ein Motor mit höherem Nennstrom läuft dann mit weniger Drehmoment.
Mikroschalter als Endstops	3	Einer für X, je einer pro Z-Seite. Nur zwei Anschlüsse (Signal und Masse) nötig.
RC-Servo, 5 V	1	Getestet mit MG90.
Kabel, kleiner Schraubendreher		Multimeter ist hilfreich, aber nicht Pflicht.
PC mit G-Code-Sender		Z. B. ioSender (Windows), alternativ gSender oder CNCjs.

1. Firmware aufspielen

1. Board per USB-C mit dem PC verbinden. Motoren und Netzteil bleiben ab.
2. Taste BOOTSEL gedrückt halten, kurz RST drücken, BOOTSEL loslassen. Am PC erscheint ein Laufwerk namens RPI-RP2.
3. `grblHAL-FYSETC-R4.uf2` auf dieses Laufwerk ziehen. Das Board startet von selbst neu, das Laufwerk verschwindet.
4. Sender starten, den neuen COM-Port wählen, verbinden und die Konsole öffnen.
5. `$I` senden. In der Antwort muss „FYSETC R4 V1.0“ stehen.

Kommt kein Laufwerk RPI-RP2: anderes USB-Kabel probieren, Tastenfolge wiederholen (siehe Fehlersuche).

2. Eingebaute Einstellungen laden

Beim Flashen bleiben schon gespeicherte grblHAL-Einstellungen erhalten, die eingebauten Werte greifen dann nicht. Damit sicher alles stimmt, sendest du einmal:

```
$RST=*
```

Das lädt die in die Firmware eingebauten Werte und setzt auch die Koordinaten-Offsets (G54 usw.) zurück. Danach `$+` senden und mit der Ausgabe im Abschnitt „Endkonfiguration“ vergleichen. Wichtig sind vor allem `$22=523`, `$130=100` und `$132=90`. Stimmen sie, ist die Firmware richtig geladen.

3. Verkabeln (alles stromlos)

Die Beschriftung direkt auf der Platine ist klein und schlecht lesbar. Nimm dieses Bild als Wegweiser: Es zeigt, welcher Stecker wo sitzt und in welcher Reihenfolge die Stifte belegt sind (Motoren B2 B1 A1 A2, Endstops SIG GND VCC, Laser-Stecker GPIO22 GND V).

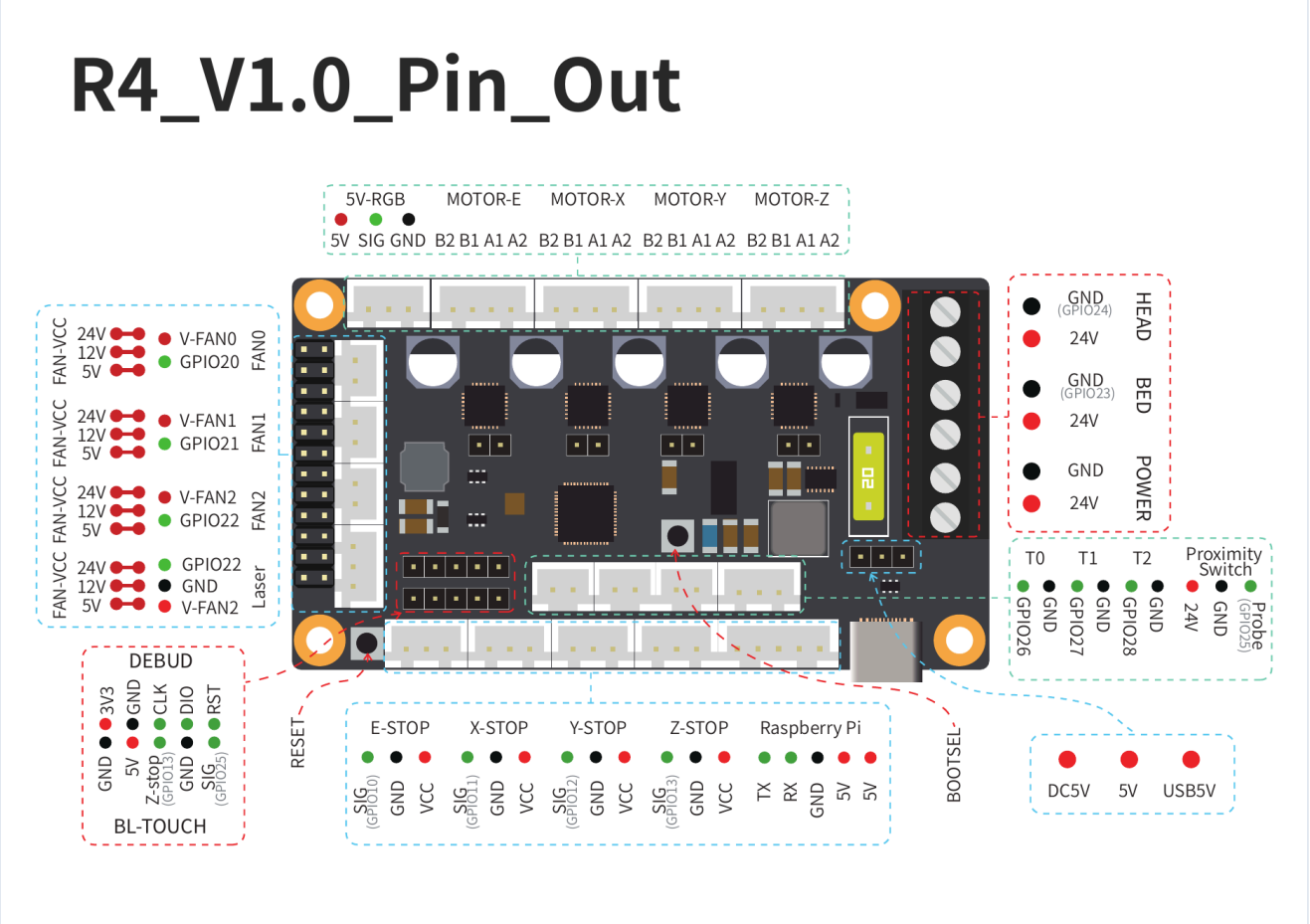


Bild: R4_PIN_OUT.pdf aus dem FYSETC-R4-Repository von FYSETC (siehe Quellen).

1. **DIAG-Jumper abziehen:** die vier Jumper an den Stiftleisten „E-Diag“, „X-Diag“, „Y-Diag“ und „Z-Diag“ neben den Treibern. Stecken sie, melden die Treiber falsche Endstops.
2. **Motoren anstecken:** X-Motor an MOTOR-X, erster Z-Motor an MOTOR-Z, zweiter Z-Motor an MOTOR-E. MOTOR-Y bleibt leer. Motoren niemals bei eingeschaltetem Netzteil an- oder abstecken, das kann die Treiber zerstören.
3. **Endstops anstecken:** X-Endstop an X-STOP. Der Endstop auf der Seite von Z-Motor 1 an Z-STOP, der auf der Seite von Z-Motor 2 an E-STOP. Endstop und Motor einer Seite müssen zusammengehören, sonst fährt beim Homing der falsche Motor weiter. Belegt werden nur Signal (SIG) und Masse (GND). Die Stiftfolge am Stecker ist SIG, GND, VCC (siehe Bild). Der Sensor-Jumper bleibt auf 3,3 V, nie auf 24 V. Y-STOP bleibt leer.
4. **Servo an den Laser-Stecker:** Spannungs-Jumper dieses Steckers auf 5 V stellen. Rot an den Plus-Pin, Braun oder Schwarz an GND, Gelb oder Weiß an Signal (GPIO22). Im Bild stehen die Stifte des Laser-Steckers von oben nach unten so: GPIO22 (Signal), GND, V (Plus). Bei Unsicherheit vor dem Anstecken messen: Der Plus-Pin zeigt 5 V gegen GND. FAN2 bleibt unbenutzt, weil er am selben Pin hängt.
5. **Netzteil** an die Schraubklemme POWER (24 V und GND, Polung prüfen). Ohne diese Spannung melden sich die Treiber nicht.

Die Anschlüsse im Überblick stehen in der Tabelle oben, die komplette Belegung im Abschnitt „Pin-Belegung FYSETC R4 V1.0“.

4. Einschalten und Treiber prüfen

1. USB anstecken, danach Netzteil einschalten und im Sender verbinden.
2. Es darf keine Meldung „Could not communicate with stepper driver!“ erscheinen. Kommt sie: Netzteil an POWER prüfen, dann Reset.
3. `$I` senden. In der Zeile NEWOPT muss `TMC=7` stehen.
4. Steht grblHAL im Alarm, hebt `$X` ihn auf.

5. Endstops prüfen

1. `$LIM` senden. Die Antwort zeigt in `[LIMITS:…]`, welche Schalter gerade ausgelöst sind. Der unbelegte Y-Eingang meldet sich dabei als ausgelöst, das ist normal.
2. Jeden Schalter von Hand drücken und erneut `$LIM` senden. X, Z und Z2 müssen getrennt auftauchen (Z im ersten Feld, Z2 im dritten).
3. Ein Schalter dauernd ausgelöst: DIAG-Jumper noch gesteckt oder Schalter ist ein Öffner. Mit Öffnern (NC) `$5=5` setzen.

6. Motoren testen

Mit der Hand am Netzschalter, die Mechanik frei. Laufen die Z-Motoren gegeneinander, verspannt sich das Portal.

1. `$J=G91 G21 X5 F300` fährt X um 5 mm. Es muss vom X-Endstop **wegfahren**. Fährt es hin, `$3` zwischen 0 und 1 umstellen (oder die Motoradern umdrehen).
2. `$J=G91 G21 Z2 F100` fährt Z um 2 mm. Beide Z-Motoren müssen gleichsinnig laufen und von den Endstops wegfahren. Laufen beide falsch herum, `$3=5` setzen (X und Z invertiert). Läuft nur der zweite Z-Motor verkehrt, `$8=4`.
3. Zuckt ein Motor nur und bleibt stehen, sind zwei Spulenpaare am Stecker vertauscht. Strom aus, zwei Adern vertauschen: erst 2 und 3, hilft das nicht zurück und 2 und 4. Genauer im Abschnitt „Motor-Adern ohne Multimeter“.

7. Referenzfahrt (Homing)

1. `$HZ` senden. Beide Z-Motoren fahren zu ihren Schaltern, jeder stoppt an seinem eigenen (Squaring). Läuft einer weiter und drückt gegen den Anschlag, sofort ausschalten und Z-STOP und E-STOP tauschen.
2. `$HX` für X, danach `$H` für alles. Die Reihenfolge ist Z, dann X.
3. Kommt `error:5`, ist Homing aus: `$22=523` setzen.
4. Nach dem Homing meldet der Status `MPos:0.000,0.000,0.000`. Mit `$#` prüfen, dass `G54` auf 0 steht. Sonst `G10 L2 P1 X0 Z0` senden, dann zeigt der Sender die Maschinenposition ohne Versatz.
5. Steht das Portal nach `$HZ` schief, den Versatz in `$172` (mm) eintragen, neu referenzieren und messen, bis es passt.

8. Verfahrweg prüfen

Nach dem Homing zählt die Maschine **positiv**: Der Nullpunkt liegt am Home-Schalter, der nutzbare Bereich ist Verfahrweg minus 2 mm Pull-off, also X 0 bis 98 und Z 0 bis 88 mm. Fahre in

Maschinenkoordinaten:

```
$J=G53G21X10F3000
$J=G53G21Z10F600
```

Dann vorsichtig zum anderen Ende (`X98` , `Z88`). Ein Ziel außerhalb, etwa `X99` , muss `error:15 - Jog target exceeds machine travel` liefern. Das sind die Soft-Limits. Stimmen die Wege nicht mit deiner Mechanik, passt du `$130` und `$132` an (siehe unten) und referenzierst neu.

9. Servo testen

Die Servo-Korrektur steckt in der Firmware vom 19.10.2026 und ist am Gerät getestet und funktioniert. Bei älterer Firmware siehe Fehlersuche, falls `M280` einen Fehler meldet.

1. `M280 P0 S90` stellt die Mitte ein. `P` ist die Servo-Nummer (0), nicht der Pin.
2. `M280 P0 S0` und `M280 P0 S180` fahren die Enden an. Brummt der Servo in einer Endlage, nimm nur den Bereich, den er sauber erreicht, etwa `S10` bis `S170` .
3. Beim Start fährt der Servo vermutlich auf 0°. Mechanik freihalten.

10. An die eigene Mechanik anpassen

Alle Werte setzt du in der Konsole, sie bleiben gespeichert. Vergleich mit `$+` .

Was	Einstellung	So geht's
Schritte pro mm	`$100` (X), `$102` (Z)	200 × Mikroschritte (16) ÷ Weg pro Umdrehung in mm. GT2 mit 20 Zähnen sind 40 mm, also 80. Spindel mit 8 mm Steigung: 400. Kontrolle: `G91 G0 X10` , nachmessen, neuer Wert = alter Wert × 10 ÷ gemessener Weg.
Verfahrweg	`$130` , `$132`	Nutzbaren Weg zwischen Home-Schalter und Gegenseite messen. Danach neu referenzieren.
Drehrichtung	`$3` (X=1, Z=4 addieren), `$8=4` für Z2 allein	Siehe Schritt 6.
Öffner statt Schließer	`$5=5`	Gilt für X und Z, auch Z2.
Geschwindigkeit, Beschleunigung	`$110` – `$112` , `$120` – `$122`	Schrittweise erhöhen, bis der Motor Schritte verliert, dann zurück.
Motorstrom	`$140` – `$142`	Etwa 70 % des Nennstroms, höchstens 958 mA (mehr gibt `error:52`). Z gilt für beide Z-Motoren.
Homing-Tempo	`$180` – `$182` fein, `$190` – `$192` grob	Z langsamer als X.

Willst du die Werte dauerhaft als Standard, damit `$RST=*` sie wiederherstellt, trägst du sie in `machine_defaults.h` ein und baust die Firmware neu (nächster Abschnitt).

11. Firmware selbst bauen und ändern

Nur nötig, wenn du Standardwerte ändern oder den Code anpassen willst. Das geht unter Linux (auch WSL unter Windows). Die Befehle stehen im Abschnitt „Optional: Firmware selbst bauen“ weiter unten. Danach:

1. Nach dem Anwenden des Patches liegt `machine_defaults.h` im Ordner `RP2040`. Dort stehen die Werte, jeweils mit Kommentar zur zugehörigen `$`-Einstellung.
2. Wert ändern, im Ordner `build` erneut `ninja` aufrufen.
3. Die neue `grblHAL.uf2` wie in Schritt 1 aufspielen (mit `$UF2` im Sender kommst du ohne BOOTSEL-Taste in den Bootloader).
4. `$RST=*` senden, damit die neuen Standardwerte geladen werden.

grblHAL steht unter der GPL-3. Gibst du die Firmware oder den Patch weiter, gilt das auch für deine Änderungen.

Geschafft, wenn

- `$I` meldet FYSETC R4 V1.0 und `TMC=7`, beim Start kommt keine Treiber-Fehlermeldung.
- `$LIM` zeigt jeden Schalter einzeln.
- X und Z fahren in die erwartete Richtung, beide Z-Motoren gleichsinnig.
- `$H` endet mit `MPos:0.000,0.000,0.000` und `G54` ist 0.
- Z steht nach dem Homing parallel.
- `X99` liefert `error:15`.
- `M280 P0 S90` bewegt den Servo.

3. Firmware flashen

Der RP2040 hat einen eingebauten Bootloader, der sich wie ein USB-Stick verhält. Du brauchst keinen Treiber und kein Flash-Tool. Die vorinstallierte Klipper-Firmware wird dabei überschrieben; zurück kommst du jederzeit auf demselben Weg.

1. **USB-C anstecken.** Für das Flashen reicht USB-Strom; Motoren und Netzteil können noch abgesteckt bleiben.
2. **Bootloader starten:** BOOTSEL-Taster gedrückt halten, kurz RST drücken, dann BOOTSEL loslassen.
3. **Datei kopieren:** Am PC erscheint ein Laufwerk **RPI-RP2**. `grblHAL-FYSETC-R4.uf2` darauf ziehen. Das Board startet danach von selbst neu, das Laufwerk verschwindet.
4. **Verbinden:** Das R4 meldet sich jetzt als serieller USB-Port (Windows: „Serielles USB-Gerät (COMx)“). Im Sender diesen Port wählen, Baudrate 115200 (bei USB eigentlich egal).
5. **Prüfen:** `$I` senden. In der Antwort muss `FYSETC R4 V1.0` stehen. Mit `$pins` siehst du die Pin-Belegung, sie sollte der Tabelle oben entsprechen.
6. **Optional auf Werkseinstellungen:** `$RST=*` setzt alle Einstellungen zurück – sinnvoll, falls vorher schon einmal grblHAL auf dem Board war.

Steht grblHAL im Alarm-Zustand (z. B. nach einem ausgelösten Endstop), `$X` hebt den Alarm auf.

4. Verdrahtung

Verdrahte nur stromlos (Netzteil aus, USB ab). Der wichtigste Punkt: Die **DIAG-Jumper müssen runter**. Laut Schaltplan verbinden sie den DIAG-Ausgang jedes TMC2209 mit seinem Endstop-Eingang. Gesteckt stören die Treiber deine Schalter. Auf deinen Fotos sehen sie gesteckt aus.

1. **DIAG-Jumper ziehen:** die vier Jumper an den Stiftleisten „E-Diag“, „X-Diag“, „Y-Diag“, „Z-Diag“ neben den Treibern.
2. **Motoren anstecken:** X-Motor → MOTOR-X, Z-Motor 1 → MOTOR-Z, Z-Motor 2 → MOTOR-E. Pinfolge laut Pinout B2 B1 A1 A2; Spulenpaare vorher mit dem Multimeter prüfen. Die Drehrichtung korrigierst du später per Einstellung.
3. **Endstops anstecken:** X → X-STOP. Der Endstop auf der Seite von Z-Motor 1 → Z-STOP, der auf der Seite von Z-Motor 2 → E-STOP. Endstop und Motor müssen auf derselben Seite sitzen, sonst fährt beim Homing der falsche Motor weiter.
4. **Nur SIG und GND belegen.** Der dritte Pin (VCC) führt je nach Jumper 3,3 V oder 24 V und ist nur für aktive Sensoren. Lass den Jumper auf 3,3 V.
5. **Pull-up prüfen:** Im Schaltplan ist der 4,7-kΩ-Pull-up als Bestückungsoption markiert. Im Test reichte ein nackter Mikroschalter nur zwischen SIG und GND ohne Zusatzteile (offen = nicht ausgelöst, gedrückt = ausgelöst). Ein nicht angeschlossener Eingang meldet dagegen „ausgelöst“. Wackelt ein angeschlossener Eingang, löte 4,7 kΩ zwischen SIG und VCC (Jumper auf 3,3 V).
6. **NO- oder NC-Schalter:** Beides geht. Die Firmware erwartet standardmäßig NO (öffnet, schließt beim Auslösen nach GND). Für NC stellst du in Kapitel 5 `$5=5` ein. NC ist sicherer, weil ein Kabelbruch dann als „ausgelöst“ erkannt wird.
7. **Servo an den Laser-Stecker:**
   - Den Spannungs-Jumper für den Laser-Stecker (3×2-Stiftleiste „24V / 12V / 5V“) auf **5 V** setzen.
   - Vor dem Anstecken messen: ein Pin muss 5 V gegen GND zeigen (Servo-Plus, rot), der Signal-Pin im Ruhezustand ca. 0 V.
   - Servo: rot → V-Pin, braun/schwarz → GND, gelb/weiß → Signal (GPIO22, 3,3-V-Logik, 100 Ω in Reihe).
   - Der 5-V-Wandler des Boards liefert bis 5 A (Sicherung 5 A), ein normaler Servo bis MG996R-Klasse (Blockierstrom ca. 2,5 A) läuft daran. Große Servos oder Betrieb mit Raspberry Pi: Plus von einem eigenen 5-V-Netzteil (BEC), nur GND und Signal ans R4.
   - **FAN2 frei lassen.** GPIO22 schaltet laut Schaltplan auch den FAN2-MOSFET, ein Lüfter dort würde im Servo-Takt flackern.
8. **Netzteil** an die Schraubklemme „POWER“ (24 V / GND). Die Treiber brauchen diese Spannung, sonst schlägt die TMC-Kommunikation fehl.

5. Einstellungen

grblHAL hat keine Config-Datei. Du sendest die Werte als `$`-Befehle, sie bleiben im Flash des Boards gespeichert. Pins, die Achszuordnung (Z doppelt, mit Squaring) und der Servo sind fest in der Firmware – einstellen musst du nur noch die Maschinenwerte. Achs-Bitmasken: X = 1, Y = 2, Z = 4 (addieren).

Das sind Startwerte für den ersten Aufbau, Zeile für Zeile in die Konsole des Senders schicken. Die Mechanikwerte sind Platzhalter, die du anpasst (Tabelle darunter). Die am fertigen Gerät getesteten Werte stehen im Abschnitt „Endkonfiguration“ weiter unten:

```
$1=255
$3=0
$5=0
$8=0
$20=0
$21=0
$22=1
$23=1
$24=50
$25=500
$27=2
$44=4
$45=1
$46=0
$100=80
$102=400
$110=3000
$112=600
$120=200
$122=50
$130=300
$132=100
$140=800
$142=800
$150=16
$152=16
$172=0
```

Danach mit `$+` kontrollieren (`$$` nimmt diese Version nicht an). Was die Werte tun und wie du sie anpasst:

Einstellung	Bedeutung	Anpassen
`$1=255`	Motoren bleiben bestromt	Wichtig für Dual-Z, sonst kann eine Seite absacken
`$3`	Drehrichtung umkehren (Maske)	Z. B. `$3=1` , wenn X falsch herum läuft
`$8`	Richtung nur des zweiten Z-Motors umkehren	`$8=4` , wenn Z2 gegen Z1 läuft (z. B. spiegelbildlich montiert)
`$5`	Endstops invertieren	0 für NO-Schalter, `$5=5` für NC an X und Z (gilt auch für Z2)
`$20` / **`$21`**	Soft- / Hard-Limits	`$20=1` erst, wenn Homing sicher funktioniert
`$22=1`	Homing aktiv. 523 schaltet zusätzlich den Maschinennullpunkt 0 und die Geschwindigkeiten pro Achse frei (siehe Endkonfiguration)	–
`$23`	Homing-Richtung negativ (Maske)	`$23=1` : X referenziert am Minimum, Z nach oben. Z-Endstops unten: `$23=5`

Einstellung	Bedeutung	Anpassen
`$24` / `$25`	Homing-Tempo fein / grob (mm/min), gilt für alle Achsen. Pro Achse getrennt nach $22=523: $180–$182 / $190–$192	Langsam anfangen
`$27`	Rückweg vom Schalter (mm). Pro Achse: $290–$292	1–3 mm
`$44=4`, `$45=1`	Homing-Reihenfolge: erst Z, dann nur X	Y bleibt draußen, weil kein Y-Endstop angeschlossen ist
`$100` / `$102`	Schritte pro mm für X / Z	200 × 16 ÷ Weg pro Umdrehung. GT2-Riemen, 20 Zähne: 80. Gewinde 8 mm Steigung: 400. TR8×2: 1600
`$110` / `$112`	Höchstgeschwindigkeit (mm/min)	Vorsichtig hochtasten
`$120` / `$122`	Beschleunigung (mm/s²)	Vorsichtig hochtasten
`$130` / `$132`	Verfahrweg (mm)	Real messen, für Soft-Limits
`$140` / `$142`	Motorstrom in mA (RMS)	Etwa 70 % des Nennstroms; Z gilt für beide Z-Motoren
`$150` / `$152`	Microsteps	Bei Änderung `$100` / `$102` mitrechnen
`$172`	Squaring-Versatz Z (mm)	Feinjustage, siehe Kapitel 6

Der **Servo** braucht keine Einstellung. Er wird mit `M280 P0 S<Winkel>` gefahren, Winkel 0–180. Die Firmware gibt dafür 544–2400 µs Pulsbreite bei 50 Hz aus (Marlin-Standard).

6. Testen

Teste in dieser Reihenfolge und halte beim ersten Fahren die Hand am Netzschalter. Laufen die Z-Motoren gegeneinander, verspannt sich das Portal.

1. **Treiber prüfen** (24 V muss anliegen): `M122` habe ich an diesem Build nicht getestet. Sicher erkennbar: Beim Start darf keine Meldung „Could not communicate with stepper driver!“ kommen, und `$I` zeigt in der NEWOPT-Zeile `TMC=7`. Sonst: Netzteil prüfen, dann Adressen oder Verkabelung.
2. **Endstops prüfen:** `?` senden, dann jeden Schalter von Hand drücken und erneut `?` senden. Im Status erscheint `Pn:X` bzw. `Pn:Z`. Beide Z-Schalter melden sich als Z – einzeln testen. Kommt das ? im Terminal nicht an, nimm $LIM: Z und Z2 erscheinen dort getrennt. Ist ein Schalter dauernd aktiv: `$5` falsch oder DIAG-Jumper noch gesteckt.
3. **Vorsichtig joggen:** `$X`, dann `$J=G91 G21 X5 F300`. X+ muss vom X-Endstop wegfahren. `$J=G91 G21 Z2 F100`: beide Z-Seiten müssen gleichsinnig fahren, Z+ in Richtung der Z-Endstops (wenn sie oben sitzen). Falsche Richtung bei beiden: `$3`. Nur Z2 falsch: `$8=4`.

4. **Einzeln homen:** erst `$HZ`, dann `$HX`, zum Schluss `$H`. Beim Z-Homing muss jeder Z-Motor an seinem eigenen Schalter stoppen. Läuft einer weiter und drückt gegen den Anschlag, sind Endstop und Motorseite vertauscht – sofort ausschalten und Z-STOP/E-STOP tauschen.
5. **Z parallel einstellen:** Steht das Portal nach `$HZ` schief, den Versatz in `$172` (mm) eintragen – positiv oder negativ –, neu homen und messen, bis es passt.
6. **Soft-Limits einschalten:** `$20=1`, sobald Homing und `$130` / `$132` stimmen.
7. **Servo:** `M280 P0 S90` (Mitte), dann `M280 P0 S0` und `M280 P0 S180`. Brummt der Servo in einer Endlage, fahr nur den Bereich, den er sauber erreicht, z. B. `S10` bis `S170`.

Im G-Code steuerst du den Servo genauso: `M280 P0 S45` z. B. zwischen zwei Fahrbefehlen. Anders als eine Achse fährt er nicht koordiniert mit X/Z, sondern springt auf den Winkel.

7. Endkonfiguration (getestet am 19.10.2026)

Diese Werte laufen auf dem aufgebauten Gerät: X mit einem Motor und Endstop, Z mit zwei Motoren, Squaring und Homing funktionieren. Ab der Firmware vom 19.10.2026 sind sie als Standardwerte eingebaut (Datei machine_defaults.h): $RST=* stellt genau diese Konfiguration her, das Abtippen ist nur noch für ältere Firmware nötig. Zum Zurückspielen (nach einem Neuflashen oder `$RST=*`) die Zeilen der Reihe nach in die Konsole senden. `$22` muss zuerst kommen, weil `$180` – `$192` erst nach dem Einschalten von „Per axis feedrates“ (Bit 9, Wert 512) existieren.

```
$22=523
$1=255
$3=1
$20=1
$23=5
$26=50
$45=1
$100=80
$102=400
$110=18000
$112=6000
$120=6000
$122=600
$130=100
$132=90
$180=150
$181=150
$182=150
$190=3000
$191=3000
$192=600
$290=2
$292=2
$481=1000
```

Alle anderen Werte stehen auf dem Standard, unter anderem `$4=7`, `$5=0`, `$8=0`, `$140` – `$142=500`, `$150` – `$152=16`, `$172=0`, `$347=5`, `$348=2.5`, `$349=25`.

Koordinaten nach dem Homing: Mit `$22=523` (Maschinennullpunkt 0) steht die Maschine nach `$H` bei MPos 0 und fährt nur in positive Richtung. Der nutzbare Bereich ist Verfahrweg minus Pull-off: X 0 bis 98

mm, Z 0 bis 88 mm. G54 muss dabei 0 sein (`G10 L2 P1 X0 Z0`, Kontrolle mit `$#`). Bleibt ein alter G54-Offset stehen, zeigt der Sender falsche Werte, weil WPos = MPos − G54 ist, zum Beispiel X=50 direkt nach dem Homing. Die Soft-Limits rechnen in Maschinenkoordinaten, die Achse scheint dann über den Verfahrweg hinauszufahren. Ein Jog in negative Richtung am Home-Schalter (`$J=G53G21Z-1`) liefert `error:15 - Jog target exceeds machine travel`. Das ist richtig, positive Jogs fahren vom Schalter weg. Nach jeder Änderung von `$130` / `$132` oder `$22` Achsen neu referenzieren.

Wert	Bedeutung
`$22=523`	Homing an (1) + einzelne Achsen mit `$HZ` / `$HX` (2) + Maschinennullpunkt 0 nach dem Homing (8) + Geschwindigkeiten pro Achse (512)
`$1=255`	Motoren bleiben dauernd bestromt (Z hält gegen die Schwerkraft)
`$3=1`	Drehrichtung von X umgekehrt
`$20=1`	Soft-Limits an. Nach jedem Einschalten zuerst homen
`$23=5`	X und Z referenzieren in positive Richtung (X=1, Z=4). Der Verfahrbereich liegt danach im Negativen
`$26=50`	Wartezeit zwischen den Homing-Phasen in ms
`$45=1`	Homing-Reihenfolge: erst Z (`$44=4`), dann nur X. Y hat keinen Endstop
`$100=80`, **`$102=400`**	Schritte pro mm: X = 200 × 16 ÷ 40 mm (GT2, 20 Zähne), Z = 200 × 16 ÷ 8 mm
`$110=18000`, **`$112=6000`**	Höchstgeschwindigkeit X / Z in mm/min. Z sind 750 U/min der Spindel, bei Schwingen oder Brummen reduzieren
`$120=6000`, **`$122=600`**	Beschleunigung X / Z in mm/s²
`$130=100`, **`$132=90`**	Verfahrweg X / Z in mm (für die Soft-Limits)
`$180` – **`$182=150`**	Homing, genaues Anfahren pro Achse in mm/min
`$190` / **`$191=3000`**, **`$192=600`**	Homing, schnelles Suchen X, Y / Z in mm/min. Wird von `$110` – `$112` gedeckelt
`$290=2`, **`$292=2`**	Rückweg vom Schalter X / Z in mm. Der Standard für X war 12 mm und machte das Homing sehr langsam
`$481=1000`	Status-Autoreport jede Sekunde. Mit `0` ausschalten, wenn das Terminal ruhig sein soll

Die komplette Ausgabe von `$+` zum Vergleich:

```
$0=5.0
$1=255
$2=0
$3=1
$4=7
$5=0
$6=0
$8=0
$9=1
```

```
$10=511
$11=0.010
$12=0.002
$13=0
$15=0
$16=0
$18=0
$19=0
$20=1
$21=0
$22=523
$23=5
$26=50
$27=1.000
$28=0.100
$29=0.0
$30=1000.000
$31=0.000
$32=0
$33=5000.0
$34=0.0
$35=0.0
$36=100.0
$37=0
$39=1
$40=0
$41=0
$42=2
$43=1
$44=4
$45=1
$46=0
$56=5.0
$57=100.0
$58=-5.0
$59=500.0
$60=0
$61=0
$62=0
$63=3
$64=0
$65=0
$100=80.00000
$101=250.00000
$102=400.00000
$110=18000.000
$111=500.000
$112=6000.000
$120=6000.000
$121=10.000
$122=600.000
$130=100.000
$131=200.000
$132=90.000
$140=500
$141=500
$142=500
$150=16
$151=16
$152=16
$172=0.000
$180=150.0
$181=150.0
$182=150.0
```

```
$190=3000.0
$191=3000.0
$192=600.0
$200=22.0
$201=22.0
$202=22.0
$210=50
$211=50
$212=50
$220=22.0
$221=22.0
$222=22.0
$290=2.000
$291=1.000
$292=2.000
$339=0
$341=0
$342=30.0
$343=25.0
$344=200.0
$345=200.0
$346=1
$347=5.0
$348=2.500
$349=25.000
$384=0
$392=4.0
$393=1.0
$394=0.0
$398=100
$481=1000
$485=0
$486=0
$539=0.0
$673=0.0
$676=15
$680=0
$703=0
```

8. Befehle im Betrieb

Diese Befehle hat die gebaute Version (`$HELP Commands`). `$$` wird hier nicht akzeptiert (`error:3`), nimm stattdessen `$+` . Das Echtzeit-Zeichen `?` kommt in manchen Terminals nicht an. Dann hilft `$LIM` oder der Autoreport `$481=1000` .

Befehl	Wofür
`$+`	alle Einstellungen ausgeben
`$140`	eine einzelne Einstellung lesen (`$140=800` setzt sie)
`$ES`	alle Einstellungen mit Name, Einheit und erlaubtem Bereich
`$I` / **`$I+`**	Version, Board, Plugins (Trinamic, Homing pulloff, PWM servo)
`$HELP`	Hilfethemen, z. B. `$HELP Dual axis` , `$HELP Homing` , `$HELP Limits`
`$PINS`	Pin-Belegung der Firmware (Kontrolle gegen die Tabelle unten)

Befehl	Wofür
`$LIM`	aktuell ausgelöste Endstops: `[LIMITS:min,max,min2,max2]`. Im Test stand Z (GPIO13) im ersten Feld, Z2 (GPIO10) im dritten
`$HSS`	Status der Homing-Schalter
`$PINSTATE`	Zustand der Hilfs-Pins
`$SDS`	Treiberstatus (bei diesem Build ohne Ausgabe, kein Fehler)
`$G` / `$#`	Parser-Zustand / Koordinatensysteme, Offsets
`$X`	Alarm löschen
`$H`, `$HZ`, `$HX`	alle Achsen / nur Z / nur X referenzieren (`$HZ` und `$HX` brauchen Bit 2 in `$22`)
`$J=G91 G21 X5 F300`	Jogging, relativ in mm
`M280 P0 S90`	Servo auf Winkel 0–180
`$RST=*`	alles auf Werkseinstellung (Löscht alle Werte!)
`$UF2`	in den UF2-Bootloader, zum Neuflashen ohne BOOTSEL-Taste
`$REBOOT`	Controller neu starten

Häufige Fehlermeldungen:

Meldung	Bedeutung
`error:3`	Befehl unbekannt, z. B. `$$`
`error:5`	Homing nicht eingeschaltet, `$22` prüfen
`error:52`	Wert außerhalb des erlaubten Bereichs, z. B. `$140` über 958 mA
`Could not communicate with stepper driver!`	TMC-Treiber antwortet nicht, meist fehlen die 24 V

9. Erkenntnisse zum Board

Was sich beim Aufbau herausgestellt hat, zum Nachschlagen.

Stromversorgung

Alle Spannungen entstehen aus den 24 V am Eingang. Die Ausgangsspannungen von 5 V und 12 V habe ich aus den Spannungsteilern im Schaltplan berechnet.

Schiene	Wandler	Max. Strom	Absicherung
24 V	direkt vom Netzteil	netzteilabhängig	Flachsicherung 20 A, TVS-Diode SMAJ28A (nicht über ca. 28 V gehen)
5 V	Silergy SY8205 (Schaltregler)	5 A, Spitze 6 A	5 A
12 V	Rychip RY9323HCT6 (Schaltregler)	2 A	3 A
3,3 V	1117-3.3 (Linearregler, aus 5 V)	ca. 0,8–1 A laut Typ, praktisch deutlich weniger	–

Der Jumper J2 wählt, ob die 3,3-V-Logik aus USB (500-mA-Sicherung) oder aus den 5 V kommt. Damit läuft das Board zum Flashen auch nur über USB. Ein Servo bis MG996R-Klasse (ca. 2,5 A Blockierstrom) läuft am 5-V-Wandler, solange kein Raspberry Pi am Board hängt.

Motorstrom

- `$140` – `$142` erlauben 17 bis **958 mA** (RMS). Mehr geht nicht, `$140=1000` gibt `error:52`. Die Grenze kommt von den 0,22-Ω-Messwiderständen: 0,325 V ÷ (0,22 Ω + 0,02 Ω) ÷ √2.
- Der zweite Z-Motor (E-Treiber) hat keine eigene Einstellung und übernimmt Z: Strom `$142`, Mikroschritte `$152`, Haltestrom `$212`.
- Die Treiber sind fest verlötet und haben keinen Kühlkörper. Dauerbetrieb nahe 958 mA braucht einen Lüfter.
- Braucht ein Motor mehr, ist das BTT SKR Pico der naheliegende Umstieg. Seine steckbaren TMC2209 haben 0,11-Ω-Messwiderstände, rechnerisch kommen sie auf etwa 1,77 A (aus den Standardwerten berechnet, nicht in den BTT-Unterlagen nachgeprüft). grblHAL unterstützt es mit einer fertigen Board-Datei.

Endstops

- Ein nackter Mikroschalter nur zwischen SIG und GND funktioniert ohne Zusatzteile: offen = nicht ausgelöst, gedrückt = ausgelöst, `$5=0`.
- Ein **nicht angeschlossener** Eingang meldet „ausgelöst“ (bei dir Y). Mit `$21=1` (Hard-Limits) würde das sofort einen Alarm auslösen. Deshalb bleibt `$21=0`. Alternativ ziehst du den Eingang über 10 kΩ auf 3,3 V (Sensor-Jumper auf **3V3**, nie auf 24 V).
- Die internen Pull-ups des RP2040 (`$18=0`) helfen hier nicht, weil zwischen Stecker und RP2040 ein Pufferbaustein sitzt (so laut Schaltplan gelesen).
- Test: `$LIM` ohne und mit gedrücktem Schalter. Beide Z-Schalter melden getrennt (Z im ersten Feld, Z2 im dritten).

Geschwindigkeit

Ein hartes Limit gibt es nicht. Die Firmware fährt nie schneller als `$110` – `$112`, auch nicht beim Homing. Die Schrittrate ist `mm/min ÷ 60 × Schritte/mm`: X mit 18000 mm/min und 80 Schritte/mm sind 24 000 Schritte/s, Z mit 6000 mm/min und 400 Schritte/mm sind 40 000 Schritte/s. Der Engpass ist das

Motormoment bei hoher Drehzahl, nicht der RP2040. Vorgehen: Wert schrittweise erhöhen, bis der Motor brummt oder Schritte verliert, dann etwa 70 % davon nehmen.

Schritte pro mm

`Schritte/mm = Schritte pro Umdrehung × Mikroschritte ÷ Weg pro Umdrehung`

Achse	Rechnung	Ergebnis
X (GT2, 20 Zähne = 40 mm)	200 × 16 ÷ 40	80
Z (Spindel 8 mm Steigung)	200 × 16 ÷ 8	400
Z (Spindel 2 mm Steigung)	200 × 16 ÷ 2	1600

Kontrolle: `G91 G0 X10`, nachmessen, `neuer Wert = alter Wert × 10 ÷ gemessener Weg`. Bei anderen Mikroschritten (`$150` – `$152`) die Schritte/mm mitrechnen.

Homing

- Ablauf: schnell zum Schalter (`$190` – `$192`), um den Pull-off zurück (`$290` – `$292`), langsam wieder hin (`$180` – `$182`), Wartezeit `$26`.
- Der Standard-Pull-off für X war 12 mm. Mit dem langsamen Anfahren (25 mm/min) dauerte das etwa 30 s. 2 mm und 150 mm/min beheben das.
- `$26` ist eine Wartezeit in ms, keine Strecke.
- Das Squaring von Z braucht keine Zusatzeinstellung. Die Sicherung `$347` – `$349` (Dual axis length fail) steht auf 5 %, 2,5 mm und 25 mm. Meldet das System einen Dual-Axis-Fehler, schau dort zuerst nach.
- Feinversatz der beiden Z-Seiten: `$172` in mm.

Motor-Adern ohne Multimeter

Zuckt ein Motor nur und bleibt stehen, sind die Spulenpaare am Stecker meist gekreuzt. Strom aus, dann:

1. Zwei Adern des losen Motorkabels blank aneinanderhalten und die Welle von Hand drehen. Spürbarer Widerstand heißt: Das ist ein Spulenpaar.
2. Alternativ Adern am Stecker durchnummerieren (1–4) und probeweise tauschen: erst 2 ↔ 3, hilft das nicht zurück und 2 ↔ 4.
3. Läuft er sauber, aber falsch herum: `$3` (Maske X=1, Y=2, Z=4) oder `$8=4` (nur Z2).

Idle-Timeout

`$1` ist die Wartezeit in ms bis zum Abschalten der Motoren, 255 heißt dauerhaft bestromt. `$37` hält einzelne Achsen bestromt (Maske, Z = 4), `$210` – `$212` setzen den Haltestrom in Prozent.

10. Pin-Belegung FYSETC R4 V1.0 (Referenz)

GPIO-Nummern des RP2040 laut Schaltplan und Pinout und Klipper-Config von FYSETC. Die grblHAL-Zuordnung unten wurde mit `$PINS` gegengeprüft. Nützlich, falls du das Board später anders belegen oder eine andere Firmware (Klipper, eigene Karte) verwenden willst.

Schrittmotortreiber (4 × TMC2209, fest verlötet, UART)

Treiber	Step	Dir	Enable	TMC-Adresse
X (MOTOR-X)	3	2	4	1
Y (MOTOR-Y)	16	15	14	2
Z (MOTOR-Z)	19	18	17	3
E (MOTOR-E)	6	5	7	0

TMC2209-UART (alle vier an einer Leitung): TX = GPIO8, RX = GPIO9. Messwiderstände 0,22 Ω. Pinfolge am Motorstecker laut Pinout B2 B1 A1 A2.

Endstops und Eingänge

Anschluss	GPIO	Bemerkung
E-STOP	10	Pufferbaustein 74LVC2G34, 10 kΩ in Reihe, DIAG-Jumper JP2
X-STOP	11	wie E-STOP, DIAG-Jumper JP4
Y-STOP	12	wie E-STOP, DIAG-Jumper JP5
Z-STOP	13	wie E-STOP, DIAG-Jumper JP6
PROBE	25	Messtaster
T0 / T1 / T2	26 / 27 / 28	Thermistor, ADC

Jeder Endstop-Stecker hat SIG, GND und VCC. VCC wird per Jumper auf 3,3 V oder 24 V gelegt. Der 4,7-kΩ-Pull-up ist im Schaltplan als „NC“ (nicht bestückt) markiert. Die DIAG-Jumper verbinden den Diagnose-Ausgang des Treibers mit dem Endstop-Eingang und müssen für mechanische Schalter gezogen sein.

Ausgänge

Anschluss	GPIO	Bemerkung
FAN0	20	MOSFET, Spannung per Jumper 24 / 12 / 5 V
FAN1	21	wie FAN0
FAN2	22	wie FAN0. Teilt sich GPIO22 mit dem Laser-Stecker (LASER_PWM)

Anschluss	GPIO	Bemerkung
Laser-Stecker	22	Spannung per Jumper 24 / 12 / 5 V, PWM-Signal
HEAD	24	Heizer-MOSFET, 24 V
BED	23	Bett-MOSFET, 24 V
5V-RGB	29	Datenleitung für NeoPixel-LEDs
UART0 (J4)	0 (TX) / 1 (RX)	Header mit 5 V, für Raspberry Pi o. Ä.

BED und HEAD zusammen sind bis 15 A ausgelegt (Sicherung am Eingang 20 A).

Belegung in dieser grblHAL-Firmware

Funktion in grblHAL	Anschluss	GPIO
X / Y / Z Limit	X-STOP / Y-STOP / Z-STOP	11 / 12 / 13
Z2 Limit (zweiter Z-Endstop)	E-STOP	10
Z2 Step / Dir / Enable	MOTOR-E	6 / 5 / 7
Spindel PWM	FAN0	20
Spindel Richtung	FAN1	21
Spindel Enable	HEAD	24
Kühlmittel (M8)	BED	23
Servo (M280 P0)	Laser-Stecker	22
Probe	PROBE	25
Analogeingänge 0 / 1	T0 / T1	26 / 27
TMC-UART	intern	TX 8 / RX 9
Serielle Schnittstelle	USB	UART1 auf GPIO 0 / 1 meldet `$PINS` zusätzlich

Nicht belegt in der Firmware: T2 (GPIO28) und RGB (GPIO29).

11. Optional: Firmware selbst bauen

Die mitgelieferte `.uf2` ist aus dem grblHAL-RP2040-Treiber (Stand 15.10.2026, Commit 2f5bc4a) plus dem Patch `grblHAL-RP2040-fysetc-r4.patch` gebaut, mit Pico SDK 2.1.1 und arm-none-eabi-gcc 13.2. Selbst bauen brauchst du nur für Änderungen, etwa andere Pulsbreiten für den Servo oder eine andere Achsaufteilung.

```
# Einmalig: Werkzeuge installieren (Ubuntu, Debian oder WSL)
sudo apt install git cmake ninja-build build-essential python3 gcc-arm-none-eabi
libnewlib-arm-none-eabi libstdc++-arm-none-eabi-newlib

# Raspberry Pi Pico SDK 2.1.1 holen
git clone --branch 2.1.1 --depth 1 https://github.com/raspberrypi/pico-sdk
git -C pico-sdk submodule update --init
export PICO_SDK_PATH=$PWD/pico-sdk

# grblHAL auf den Stand bringen, auf den der Patch passt, und Patch anwenden
# (grblHAL-RP2040-fysetc-r4.patch liegt im selben Ordner wie dieser Aufruf)
git clone https://github.com/grblHAL/RP2040.git
cd RP2040
git checkout 2f5bc4a
git submodule update --init --recursive
git apply ../grblHAL-RP2040-fysetc-r4.patch

# Bauen
mkdir build && cd build
cmake -G Ninja -DPICO_BOARD=pico ..
ninja
# Ergebnis: build/grblHAL.uf2
```

Die Befehle ab dem Pico SDK sind am 19.10.2026 unter Ubuntu 24.04 durchgelaufen, die Werkzeuge waren dort schon installiert. Die entstandene Datei war bytegleich mit der mitgelieferten `.uf2`. Der Commit `2f5bc4a` ist wichtig, weil der Patch auf einen neueren grblHAL-Stand nicht mehr passen muss.

Der Patch verändert diese Stellen:

- `boards/fysetc_r4_map.h` (neu): Pin-Belegung, TMC2209 über UART1 auf GPIO 8/9, Messwiderstand 220 mΩ, Servo auf GPIO22
- `boards/fysetc_r4.c` (neu): Board-Start
- `tmc_uart.c`: Treiber-Adressen X=1, Y=2, Z=3, E=0
- `driver.h`, `CMakeLists.txt`: Board eingetragen
- `my_machine.h`: Board gewählt, `Z_GANGED` + `Z_AUTO_SQUARE`, `PWM_SERVO_ENABLE`, `HOMING_PULLOFF_ENABLE`

Die Servo-Pulsbreiten stehen in `plugins/pwm_servo_m280.c` (`DEFAULT_MIN_PULSE_WIDTH` / `DEFAULT_MAX_PULSE_WIDTH`).

Seit dem 19.10.2026 korrigiert der Patch außerdem zwei Fehler in `ioports_analog.c` des Upstream-Treibers, ohne die der Servo nicht läuft:

- Seit Commit 947338d (15.10.2026) reserviert die Firmware den PWM-Kanal beim Start. Das Servo-Plugin konnte ihn danach nicht mehr auf 50 Hz umstellen, meldete sich nicht an und `M280` antwortete mit `error:39`. Der Patch erlaubt das erneute Konfigurieren eines Pins, der schon PWM-Ausgang ist.
- Bei 200 MHz Systemtakt passt die 50-Hz-Periode mit Teiler 50 nicht in den 16-Bit-Zähler (80 000 Takte), der Servo hätte etwa 276 Hz bekommen. Der Patch erhöht den Teiler, bis die Periode passt (bei 50 Hz: Teiler 62, 64 516 Takte, 544–2400 µs).

Die Endkonfiguration steckt als Standardwerte in `machine_defaults.h` im Hauptordner des Treibers. `CMakeLists.txt` bindet die Datei in jede C-Datei ein, sobald sie vorhanden ist. Damit das vollständig geht, ändert der Patch zwei Kleinigkeiten außerhalb des Treibers:

- `grbl/settings.c` (Kern): Standardwerte für die Homing-Geschwindigkeiten pro Achse (`DEFAULT_X_HOMING_FEED_RATE` usw.) und für das Bit „Per axis feedrates“. Ohne die Datei bleibt alles wie bisher.
- `plugins/homing_pulloff.c`: Das Plugin legte den X-Pull-off fest auf 12 mm, jetzt nur noch, wenn nichts anderes vorgegeben ist.

Andere Werte änderst du in `machine_defaults.h` und baust neu. Ein Neuflashen überschreibt gespeicherte Einstellungen nicht, erst `$RST=*` lädt die Standardwerte.

12. Fehlersuche

Symptom	Wahrscheinliche Ursache	Lösung
Laufwerk RPI-RP2 erscheint nicht	Reihenfolge der Tasten, Ladekabel	BOOTSEL halten, RST drücken, BOOTSEL los; anderes USB-C-Kabel. Aus laufendem grblHAL: `$UF2`
Kein COM-Port nach dem Flashen	Falsche Datei oder Kabel	`.uf2` erneut kopieren, anderes Kabel; Geräte-Manager prüfen
Beim Start „Could not communicate with stepper driver!“, Motoren stromlos	Keine 24 V, Treiber nicht erreichbar	Netzteil an POWER, dann Reset
`M280 P0 S90` gibt `error:39`	Firmware vor dem 19.10.2026: Servo-Plugin bekommt den PWM-Pin nicht	Firmware vom 19.10.2026 flashen. `P` ist die Servo-Nummer (0), nicht der GPIO
Endstop dauernd ausgelöst	DIAG-Jumper gesteckt, `$5` falsch oder Eingang nicht angeschlossen	Jumper ziehen, `$5` umstellen, Schalter anschließen
Endstop reagiert zufällig	Eingang in der Luft, kein Pull-up	Kabel prüfen, ggf. 4,7–10 kΩ nach 3,3 V nachrüsten
Ein Z-Motor drückt gegen den Anschlag	Endstop und Motorseite vertauscht	Z-STOP und E-STOP tauschen
Z-Motoren laufen gegeneinander	Z2 spiegelbildlich montiert	`$8=4`
Motor zuckt nur und bleibt stehen	Spulenadern am Stecker gekreuzt	Adern 2 ↔ 3 tauschen, sonst 2 ↔ 4 (Strom aus)
`error:5` bei `$HZ`	Homing nicht eingeschaltet	`$22=3` bzw. `515`
`error:52` bei `$140`	Strom über 958 mA	höchstens `$140=958`
`error:3` bei `$$`	Befehl in dieser Version nicht vorhanden	`$+` benutzen

Symptom	Wahrscheinliche Ursache	Lösung
`?` liefert keine Statuszeile	Terminal sendet das Echtzeit-Zeichen nicht	`$LIM` oder `$481=1000`
Homing von X extrem langsam	Pull-off `$290` auf 12 mm und 25 mm/min beim Anfahren	`$290=2`, `$180=150`
Alarm beim Homing von Y	Y ist in einem Homing-Zyklus	`$45=1` (nur X)
Alarm sofort nach `$21=1`	Nicht angeschlossener Eingang (Y) meldet „ausgelöst“	`$21=0` oder Y-Eingang über 10 kΩ auf 3,3 V ziehen
Motoren werden sehr heiß	Strom zu hoch	`$140` / `$142` senken, Lüfter
Servo zittert	Schwache Versorgung, keine gemeinsame Masse	Eigenes 5-V-Netzteil, GND verbinden
Lüfter an FAN2 flackert	GPIO22 ist auch das Servo-Signal	FAN2 nicht benutzen
Nach dem Homing zeigt der Sender z. B. X=50, Achse fährt scheinbar über den Verfahrweg	Alter G54-Offset (WPos = MPos − G54), Soft-Limits gelten in Maschinenkoordinaten	G10 L2 P1 X0 Z0 senden, mit $# prüfen, dann neu referenzieren

13. Quellen

- FYSETC-R4 Repository – Schaltplan, Pinout, Klipper-Config
- FYSETC-R4 README – Flashen per BOOTSEL
- grblHAL RP2040-Treiber
- grblHAL Wiki – Einstellungen
- ioSender
- Silergy SY8205 – Datenblatt-Infos (Compel)
- Rychip Produktkatalog 2022 Q3 (RY9323)